Ders 25 Genişletilmiş Notu: Dönem Projesi (Kızma Birader) ve Canlı C Kodlama: Kelime Karşılaştırma

Ders 25 Genişletilmiş Notu: Dönem Projesi (Kızma Birader) ve Canlı C Kodlama: Kelime Karşılaştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Çözümlerde Sık Görülen Programlama Hataları
- 2. Dönem Projesi: Kızma Birader Oyununun Kuralları
- 3. Proje İçin Tasarım: Veri Yapıları ve Modüller
- 4. Okunabilir Kod Yazmak
- 5. Canlı Kodlama 1: İki Kelimenin Sözlükteki Sırası
- 6. Canlı Kodlama 2: Kelimelerin Sütunlarını Yazdırmak

1. Çözümlerde Sık Görülen Programlama Hataları

Hoca derse, öğrenci çözümlerinde en çok gördüğü hataları sayarak başladı. Bunların her biri, dönem boyunca üzerinde durulan bir programlama alışkanlığıyla ilgilidir.

- **Sütun silme:** Bir matristen sütun silinmesi istendiğinde bazı çözümlerde sütun silinmemiş, yalnızca sıfırlanmıştı. Bazılarında ise silme yapılmış ama sıfırlama yapılmamıştı.
- **Akış diyagramı notasyonu:** Derste gösterilen notasyonun dışında, kişisel şekillerle çizim yapılmıştı. Hocaya göre farklı bir çizim kullanılacaksa bu, en azından uluslararası standartlara uygun, yaygın bir kullanım olmalıdır. Kendi kafanıza göre şekil üretemezsiniz.
- **`pow` fonksiyonu:** Kullanılmaması istendiği hâlde kullanılmıştı. Bu işin `for` ile yapılması gerekir.
- **Döngü seçimi:** `for` kullanılması gereken yerde `while`, `while` kullanılması gereken yerde `for` kullanılmıştı.

> **Hocanın vurgusu:** Bu kurallar şu an biraz zorlayıcı gelebilir, ama uzun vadede geri dönüşü olur. Bu kurallara şimdiden alışmak, bölümün diğer derslerini çok rahatlatır.

2. Dönem Projesi: Kızma Birader Oyununun Kuralları

2.1 Oyun ve Kural Çeşitliliği

Dönem projesinde bir Kızma Birader oyunu gerçekleştirilecektir. Hocanın anlattığına göre oyun aslında Hint kökenlidir. Zaman içinde her ülke kendi kurallarını koymuş, birbirine benzeyen oyunlar üretmiştir. Proje dokümanında üç farklı ülkedeki adları ve bir bağlantı verilmiş, kuralların bir kısmı o bağlantıdan alınmıştır.

Kurallar kaynaktan kaynağa değiştiği için öğrencilerde bir karışıklık oluşmuştu. Örneğin bir zar mı, iki zar mı atılacağı versiyona göre değişebilir. Projede esas alınan, dokümandaki temel kurallardır.

2.2 Temel Kurallar

- Her oyuncunun 4 piyonu vardır.
- Oyun 2, 3 veya 4 oyuncuyla oynanır.
- Her turda üzerinde 1'den 6'ya kadar sayı olan bir zar atılır.
- Bir piyon, bulunduğu ev alanından ancak zar 6 geldiğinde çıkabilir.
- Her hücrede yalnızca bir piyon bulunabilir. Oyuncu, tahtada istediği kadar piyon bulundurabilir.
- 6 atan oyuncu iki seçenekten birini kullanır: tahtadaki bir piyonunu 6 ilerletir ya da yeni bir piyonu oyuna sokar.
- Yolun üzerindeki rakip veya kendi taşının, zar tam onun üzerine getirmiyorsa üzerinden geçilebilir.
- Zar piyonu bir rakip taşın bulunduğu hücreye getiriyorsa rakip taş evine, kendi yerine döner.
- Zar piyonu oyuncunun kendi piyonunun bulunduğu hücreye getiriyorsa o piyon hareket ettirilemez. Hocanın örneği: 3 atıldı ve 3 hücre önde kendi piyonu var. Piyonu oynatamaz, çünkü iki piyon aynı yere gelemez.
- Hedefe ulaşan piyonlara artık dokunulamaz.
- 4 piyonunu ilk hedefe ulaştıran oyuncu oyunu kazanır.

2.3 Tahta, Başlangıç Noktaları ve Hedef

Hoca dokümandaki tahtayı ekranda gösterdi. Öğrencilerin sorduğu dördüncü piyonun yeri için, hedef bölgesindeki yerleri tek tek sayarak bir, iki, üç ve dördüncü piyon son noktaya geldiğinde oyunun biteceğini anlattı.

Her rengin ayrı bir başlama noktası vardır. Bu noktalar tahtada işaretlenmemiştir, ama tasarımda unutulmamalıdır. Oyunun başında kaç oyuncu oynayacağı sorulmalıdır.

2.4 Program Komutlarla Çalışır

Soru: Oyun terminalden mi çalışacak, yoksa tuşları olan bir Windows programı gibi mi olacak? **Cevap:** Arayüz istenmiyor. Program çalıştırıldığında komutlar üzerinden işler, yani menüden ilgili değerler seçilerek işlem yapılır.

Örneğin menüde bir zar atma seçeneği olur. Zar atılınca sayı gelir. Piyon gitmesi gereken yere ilerlemeden önce oyuncuya sorulması gerekir. 6 geldiyse "Oyuna piyon mu sokmak istersin, ilerletmek mi?" diye, 5 geldiyse ve sahada birden fazla piyon varsa "Hangi piyonu oynatmak istersin?" diye sorulur.

Her aksiyondan sonra ekrana bir mesaj yazdırılmalıdır: sıranın kimde olduğu, bir taşın yendiği, yenilen taşın eve dönmesi gerektiği gibi.

Soru: Oyuncuların renkleri rastgele mi verilecek, oyuncular mı seçecek? **Cevap:** Bu serbesttir. Hoca bazı noktaları özellikle kurala bağlamadı. Amacı öğrencinin neler katabileceğini görmek ve onu esnek bırakmaktır.

Problem çok fazla kural barındırdığı için hoca iki yol gösterdi:

- **Doğru olan:** Bütün kuralları öngörerek tasarımı baştan yapmak. Buna ayrılacak vakit varsa tasarım tamamen yapılmalıdır.
- **Artımlı yol:** Önce temel kuralları kurmak. Örneğin önce bir piyonu hareket ettirebilecek noktaya gelmek, sonra bunu birden fazla piyona genişletmek.

2.5 Hedefe Tam Denk Gelmek

Soru: Bir piyon en son noktaya yerleşti. Arkadan gelen piyona aynı hücreyi gösteren zar gelirse ne olur? O el atlanır mı? **Cevap:** Hoca bu kuraldan emin olmadığı için dokümana yazmadığını söyledi ve soruyu sınıfa açtı. Sınıfta hatırlanan kural şuydu: Piyonun boş bir hedef hücresine tam denk gelmesi beklenir. Bir öğrenci, ikinci hedef hücresine de denk gelinebileceğini, arada boşluk kalabileceğini, ama o boşluğun da sonra doldurulması gerektiğini ekledi.

Hocanın kararı: Kural buysa bu şekilde uygulanır. Zorlanan olursa, piyonun son noktaya (Final Destination) ulaşmasıyla işinin bittiği kabul edilebilir. Yeter ki diğer kısımlar doğru kurulsun.

2.6 Opsiyonel Kurallar

Soru: Kaynaklarda çift zar, ya da belli sayıyla evden çıkma gibi kurallar var. Wikipedia'daki blockade kuralında iki aynı renk piyon aynı hücreyi işgal edebiliyor. Bu, "bir hücrede en fazla bir piyon" kuralıyla çelişiyor. Ne yapmalıyız? **Cevap:** Dokümandaki kurallar temel alınır ve değiştirilmez. Bunları bozmayan ek seçenekler katılabilir, bozanlar katılmaz.

Hoca izin verilen eklere üç örnek verdi:

- Oyun çabuk bitsin diye zar sayısını ikiye çıkarmak. Ama evden çıkış yine 6 ile olmalıdır.
- Güvenli noktalar (safe place): Bu noktalardaki piyon yenemez.
- Başa döndüren noktalar: Bu noktalara gelen piyon en başa döner.

Soru: Akış diyagramı isteniyor mu? **Cevap:** Hayır. Dokümandaki bu ifade hocanın hatasıydı. Bütün programın akış diyagramı beklenmiyor. İhtiyaç duyan, kendisi için çizebilir.

3. Proje İçin Tasarım: Veri Yapıları ve Modüller

3.1 Rotayı Tek Bir Yol Olarak Görmek

Hoca, tasarımda bir uyarı yaptı. Tahtadaki rotayı matrisin kaçıncı satır ve sütununa geldiğini kontrol ederek izlemek yerine, onu tek bir uzun rota olarak görmek mümkündür. Her adımın kaçıncı adım olduğu hesaplanır ve bu adımın bilgisi bir matriste saklanır.

Hocanın örneğinde başlangıç noktası 1. adımdır:

Rota adımı	Matris konumu
1	(4, 1)
2	(4, 2)
3	(4, 3)
4	(4, 4)
5	(3, 4)

Bu bir eşleme (mapping) yaklaşımıdır. Öbür türlü çok fazla kontrol gerekir: buraya geldi aşağı gidecek, buraya geldi sağa dönecek gibi.

> **Ek bilgi (derste anlatılmadı):** Hocanın beş adımı C'de büyük olasılıkla iki sütunlu bir tamsayı matrisiyle tutulabilir: `int rota[5][2] = {{4,1},{4,2},{4,3},{4,4},{3,4}};`. Dizinler 0'dan başladığı için `adim`. adımın konumu `rota[adim-1][0]` ve `rota[adim-1][1]` olur. Bu tablo `gcc -Wall` ile uyarısız derlendi ve yukarıdaki beş konumu verdi. Piyon ilerletmek bu durumda büyük olasılıkla yalnız adım numarasına zarı eklemek demektir, yön kontrolü gerekmez.

Soru: Rota için ayrı bir matrise bilgi koyma fikrini biraz daha açabilir misiniz? **Cevap:** "Koyun" diye bir şey yok, koyabilirsiniz. Hoca bir çözüm açıklamak istemediğini, yalnızca onlarca `if` kontrolü yerine farklı çözümler olabileceğini göstermek istediğini söyledi. Bunu düşünmek öğrencinin işidir ve kendini böyle geliştirir.

Hoca bu noktayı başka bir örnekle bağladı. Bir karakterin belli bir aralıkta olup olmadığını anlamak için tek tek A'ya eşit mi, B'ye eşit mi, C'ye eşit mi diye bakan çok çözüm görmüştü. Onun yerine aralığın iki ucuyla, A'dan büyük mü ve F'den küçük mü diye kontrol etmek yeterlidir.

> **Hocanın vurgusu:** Bu tarz çok fazla kontrolden mümkünse kaçınmak lazım. Tahtayı görüp hemen koda dalmak yerine önce üzerine düşünerek tasarım yapın.

3.2 Kütüphaneler

Soru: Kütüphane sınırlaması var mı? **Cevap:** Yok. İstenen kütüphane kullanılabilir.

Tahtanın görselleştirilmesi için de farklı kütüphaneler vardır, renklendirme de mümkündür. Bu bir zorunluluk değildir, ama yapmak öğrenciyi geliştirir.

3.3 Hücrelerde Çok Karakterli Etiketler

Soru: Tabloda "Home Blue 1" gibi çok harfli etiketler var, ama bir hücreye tek karakter gelebiliyor. Bunlar nasıl gösterilecek? **Cevap:** Bu size kalmış. Farklı çözümler vardır:

- Üç hücreyi yan yana birleştirip etiketi öyle yazmak.
- Piyonlara sayılar vermek. Hocanın örneği (üzerinde çok düşünmediğini belirterek):

Renk	Piyon kodları
Sarı (Yellow)	1, 2, 3, 4
Kırmızı (Red)	11, 12, 13, 14
Yeşil (Green)	111, 112, 113, 114
Mavi (Blue)	1111, 1112, 1113, 1114

Hangi yol seçilirse seçilsin, gösterim anlaşılır olmalı ve belgelenmelidir.

> **Ders sonrası not:** Hoca bu sayısal kodları, hücreye tek karakter konabildiğini söyledikten sonra önerdi. Bu kodlar büyük olasılıkla `char` matrisine sığmaz. Denemede `char` diziye konan 1111 için `gcc -Wall` taşma uyarısı verdi ve değer 87 oldu; `int` dizide 1111 korundu. Sayısal kod kullanılacaksa tahta büyük olasılıkla `int` matrisi olarak tutulmalıdır.

3.4 Zar ve Oyunun Sonu

Soru: Zar atma nasıl yapılacak? **Cevap:** Rastgele sayı üreten fonksiyon (random) kullanılacak. Her turda 1 ile 6 arasında bir sayı üretilir.

> **Ek bilgi (derste anlatılmadı):** C'de bu büyük olasılıkla `stdlib.h` içindeki `rand` ile yapılır: `zar = rand() % 6 + 1;`. `% 6` işlemi 0 ile 5 arasında bir değer verir, `+ 1` bunu 1 ile 6 arasına taşır. Program her çalışmada aynı sayı dizisini üretmesin diye başta büyük olasılıkla bir kez `srand(time(NULL));` çağrılır.

Soru: Bir oyuncu kazanınca diğerleri devam edecek mi? **Cevap:** Hayır, oyun biter.

3.5 Modüler Programlama

Hoca, projeye konsantre bir tasarımla başlanması, implementasyona ondan sonra geçilmesi gerektiğini söyledi. Hangi modüllerin gerektiği sorusu buradan doğar.

Problemleri çözerken büyük bir problemi alt problemlere ayırmak ve alt problemleri çözerek bütüne ulaşmak, çözümü genelde kolaylaştırır. Bu projedeki alt problemlerin hepsi birer modül olabilecek niteliktedir:

- **Rastgele sayı üretme:** Her turda çalışır.
- **Menü:** Oyun bitene kadar tekrar tekrar gelir. Kullanıcı bir sonraki adımda ne yapmak istiyorsa onu seçer.
- **Oyuncu sayısını seçme:** Bir kez çalışır, sonra bir daha sorulmamalıdır. Yine de bir modüldür.
- **Zar sonucuna göre karar:** Gelen sayıya göre senaryolar kurulur. 6 geldiyse iki seçenek vardır: yeni piyon çıkarmak ya da oyundaki bir piyonu ilerletmek. İlerletilecekse sahadaki hangi piyon olduğu sorulur.
- **Piyon durumu:** Piyonun tahtada mı, evde mi olduğu bir yerde tutulmalıdır. Tahtada olmayan bir piyon yalnızca oyuna eklenebilir, hareket ettirilemez.
- **Hedef kontrolü:** Piyonun son noktaya varıp varmadığı.
- **Başlangıç noktası:** Her rengin başlangıcı farklıdır.
- **Hücre kontrolü:** Gidilen noktada kayıtlı bir piyon var mı? Rakibe aitse o piyon eve döner, oyuncuya aitse oraya gidilemez.

Modül modül tasarım hem problemi daha rahat çözdürür hem de bir hata çıktığında hata ayıklamayı kolaylaştırır. İleride yazılan kodlar fonksiyon olarak tekrar tekrar çağrılabilir.

Soru: Modül derken ne kastediliyor, ayrı dosyalar mı? **Cevap:** Hayır. Modül, C programlamada fonksiyona denk düşer. Android programlamada yapılan işin ekranına, Java'da pencerelere

denk düşer gibi düşünülebilir. Sınıftan gelen örnek: Zar atmayı her seferinde kodlamak yerine bir kez fonksiyon olarak kodlar, her seferinde çağırırsınız. Modüler programlama budur.

Öğrenciler henüz C'de fonksiyon yazmayı görmedi, bu yüzden hoca fonksiyon yazılmasını beklemiyor, ama yazan olursa kabul ediyor. Önemli olan neye ihtiyaç olduğunu belirlemektir.

Soru: Tek dosya zorunluluğu var mı? **Cevap:** Evet. Başlık dosyası (header file) kullanılabilir, ama tek bir kaynak dosya (source file) olmalıdır.

3.6 Hatalı Girişleri Önlemek

Yanlış girişler için durumlar (case) kontrol edilmelidir. Hocanın verdiği bariz örnekler: 5 oyuncuyla oynanmak istenememeli, zar 7 gelememelidir. Program, kullanıcının yapabileceği bu tarz hataları eleyecek bir yapıda olmalıdır.

3.7 Başlama Sırası ve Oyuncu Sırası

Soru: Tahtanın dizilişi BRYG diye veriliyor: sol üst mavi, sağ üst kırmızı, sağ alt sarı, sol alt yeşil. Rastgele kırmızı başlarsa sıra kırmızı, sarı, yeşil, mavi diye mi gider, yoksa başlangıçta herkes sıralanıp o sırayla mı oynanır? **Cevap:** Oyunculara başta zar attırılabilir, en büyükten en küçüğe başlama sırası belli olur ve her tur bu sırayla oynanır. Ya da başlama sırası kullanıcıya girdirilebilir. İkisi de olur.

Bir öğrenci, başta atılan zarda en küçük çıkanın başladığını hatırladığını söyledi. Hoca, oyunlarda genel olarak en büyüğün başladığını düşündüğünü, bunun kuralları pek etkilemediğini belirtti.

3.8 Tahtayı Göstermek ve Önceki Pozisyonu Saklamak

Soru: Tahta oyuncu elini bitirdikten sonra mı gösterilecek? **Cevap:** Hayır. Tahta her adımda, her aksiyonda görülmelidir.

Bunun bir sebebi hata ayıklamadır. Hoca tahtanın bir önceki pozisyonunun da saklanmasını önerdi. Böylece bir şey olduğunda geri gidip piyonun nerede olduğu ve nereye geldiği gösterilebilir. Yapılanın doğru çalışıp çalışmadığı ancak böyle kontrol edilebilir.

4. Okunabilir Kod Yazmak

Soru: Değişkenlere hep ayrı anlamlı adlar mı vermek, yoksa genel şeyler için X, Y, Z mi kullanmak daha iyi? **Cevap:** Değişken adlarını doğru belirlemek kodun okunabilirliği için çok önemlidir. Çok ara değişkenler dışında, değişkenlerin hepsi amacına yönelik adlandırılmalıdır. Çok uzun adlar gerekmez. Geçici bir değişkense X, Y, Z o an için bir çözüm olabilir. Bir amaca hizmet ediyorsa adı verilmelidir.

İkinci öneri açıklama satırlarıdır. Kodda hangi kısmın ne yaptığı hem yazan hem de kontrol eden için kritiktir. Fonksiyon kullanılmıyorsa fonksiyon adları da olmayacağından, açıklama satırları daha da gereklidir.

> **Hocanın vurgusu:** Değişkenlerin anlamlı olması, açıklama satırları, kodların hizalı yazılması, belli boşluklar bırakılması çok önemlidir. İç içe geçmiş, hizalanmamış kodun okunması çok zordur.

5. Canlı Kodlama 1: İki Kelimenin Sözlükteki Sırası

Aradan sonra hoca çevrimiçi bir derleyicide öğrencilerle birlikte kod yazdı. İlk problem, iki kelimeden hangisinin sözlükte önce geldiğini bulmaktır. Hoca bunu ödevin küçük bir parçası gibi düşünmeyi önerdi.

5.1 Kelimeleri Okumak

Kelimeler dışarıdan `scanf` ile, `%s` biçimiyle okunur. Aynı tipteki değişkenler virgülle arka arkaya tanımlanabilir.

Soru: Girişte `&` işaretini unutmadınız mı? **Cevap:** Unutulmadı, böyle yazılması gerekir. `%s` ile okurken dizi adının önüne `&` konmaz. Bunun başka bir doğru yazımı `&kelime[0]`'dır. Bir dizinin adı, kendisinin ilk gözünün adresini ifade eder. Bu yüzden iki yazım birebir aynıdır.

5.2 Algoritmayı Birlikte Bulmak

Bir öğrenci önce kelimelerin uzunluğuna bakmayı önerdi. Hoca bunu örnekle reddetti: Sözlükte önce ilk harflere bakarız. Eşitse ikinci harfe geçeriz. Örneğin `ali` ile `ahmet` karşılaştırılırken ilk harfler eşittir, ikinci harflerde `h`, `l`'den küçüktür, o hâlde `ahmet` önce gelir.

İkinci soru, biri diğerinin başı olan kelimelerdir: `ali` mi `alibaba` mı önce gelir? `ali` önce gelir. Çünkü ilk üç harf eşit olduktan sonra `ali` kelimesinde null karakteri vardır ve null karakteri gelen harften her zaman küçüktür.

Soru: Karşılaştırmayı uzunlukları bularak yapmak mantıklı olmaz mı? **Cevap:** Hayır, uzunluklar üzerinden ilerlenemez. Hoca, tek harflik `B` gibi bir kelime konabileceğini söyleyerek bunu gösterdi: Kısa olmak önce gelmek demek değildir.

Sınıfla birlikte döngünün koşulları çıkarıldı. Harfler eşit olduğu sürece bir sonraki harfe geçilir ve bu, kelimelerden herhangi birinde null karakteri görülene kadar sürer. Döngünün içinde yalnızca `i` bir artırılır, döngüden önce de `i` sıfırlanır.

Döngüden çıkılınca, durduğumuz konumdaki iki karakter karşılaştırılır. Üç durum vardır: `kelime[i]` küçükse birinci kelime, eşitse kelimeler aynı, kalan durumda ikinci kelime önce gelir. Hoca üçüncü durumun artık kontrol gerektirmediğini, `else` ile yazılabileceğini belirtti. Sınıf ilk önce eşitlik durumunu unutmuştu, hoca bunu "bir şey unuttuk" diyerek buldurdu.

5.3 Kod

Aşağıdaki kod, dersteki son hâldir.

```c
#include <stdio.h>

int main()
{
    char kelime[100], kelime2[100];
    int i;

    printf("Birinci kelimeyi giriniz\n");
    scanf("%s", kelime);
    printf("Ikinci kelimeyi giriniz\n");
    scanf("%s", &kelime2[0]);

    i = 0;
    while (kelime[i] != NULL && kelime2[i] != NULL && kelime[i] == kelime2[i]) {
        i++;
    }

    if (kelime[i] < kelime2[i]) {
        printf("%s kelimesi sozlukte %s kelimesinden once gelir\n", kelime, kelime2);
    } else if (kelime[i] == kelime2[i]) {
        printf("Kelimeler aynidir\n");
    } else {
        printf("%s kelimesi sozlukte %s kelimesinden once gelir\n", kelime2, kelime);
    }
    return 0;
}
```

Hoca ilk çalıştırmada kodu birkaç sözdizimi hatasından temizledi, sonra üç durumu denedi. Adım adım izlemesi şöyledir (ara adım eklendi):

Girdi	Durulan i	kelime[i]	kelime2[i]	Sonuç
ali, ahmet	1	l	h	l büyük, ahmet önce gelir
ali, alibaba	3	null	b	null küçük, ali önce gelir
ali, ali	3	null	null	Eşit, kelimeler aynı

Bu üç sonuç, yukarıdaki kod derlenip çalıştırılarak da doğrulandı.

> **Ek bilgi (derste anlatılmadı):** Karşılaştırma karakter kodları üzerinden yapıldığı için sonuç büyük olasılıkla ancak harflerin hepsi aynı büyüklükteyse gerçek sözlük sırasını verir. Denemede `Ali` ile `ali` için program `Ali` önce gelir dedi, çünkü `A` kodu (65) `a` kodundan (97) küçüktür. Türkçe harfler de büyük olasılıkla bu sıralamanın dışında kalır.

5.4 Çıktıyı Güzelleştirmek

Öneri (öğrenciden): Mesajda "birinci kelime" demek yerine kelimenin kendisi `%s` ile yazdırılabilir. Hoca bunu hemen uyguladı: `%s kelimesi sozlukte %s kelimesinden once gelir` mesajına `kelime, kelime2` argümanları verildi. Diğer dalda sıra değiştirilerek `kelime2, kelime` yazıldı. "Kelimeler aynıdır" mesajında da aynı şey yapılabilir.

Öneri (öğrenciden): Girdi soruları `\n` ile alt satıra geçirilerek daha düzgün yapılabilir.

Biçimli yazdırmanın diğer araçları da hatırlatıldı:

- `\t`: Tab karakteri kadar sağa kaydırır. Bir öğrenci beş boşluk olduğunu söyledi, hoca tam kaç boşluk olduğunu bilmediğini belirtti.
- `%3d`, `%5.2f` gibi biçimler: Genişlik ve noktadan sonraki basamak sayısı verilebilir.
- Okunabilirlik için boşluklar: Hoca kodunda okuma kısmını, döngüyü ve döngüden sonraki kısmı bilerek bloklara ayırıp hizaladığını gösterdi.

5.5 NULL ile Karşılaştırma Uyarısı

Soru: Derleyicideki "comparison between pointer and integer" uyarısının sebebi nedir? **Cevap:** Sebep, null ile yapılan karşılaştırmadır. Bu bir hata değildir, kullanılabilir. Bunun yerine şunlar yazılabilir:

- `kelime2[i] != 0`
- `kelime2[i] != '\0'`
- Yalnızca `kelime2[i]`

Null yazmak hatalı değildir, ama null tanımlanmamış ortamlar olabilir. Orada önce tanımlanması gerekir. Null önceden tanımlıysa önce `#undef` ile kaldırılıp yeniden tanımlanabilir:

```
#undef NULL
#define NULL 0
```

> **Ders sonrası not:** Hoca tanımı söylerken "define null eşit 0" dedi; doğrusu büyük olasılıkla eşittirsiz `#define NULL 0`'dır. Denemede `#define NULL = 0` yazınca kod derlenmedi (`expected expression before '='`). Eşittirsiz sürüm ve yukarıdaki üç yazım `gcc -Wall` ile uyarısız derlendi. Hocanın `NULL`'lu kodu ise iki `comparison between pointer and integer` uyarısı verdi ve üç durumda aynı sonucu üretti. Uyarının asıl sebebi büyük olasılıkla `NULL`'ın bir işaretçi sabiti olarak tanımlı olması, karşılaştırılan `char` değerinin ise tamsayı olmasıdır.

6. Canlı Kodlama 2: Kelimelerin Sütunlarını Yazdırmak

6.1 Problem

İkinci problemde bir matrisin satırlarına kelimeler okunur. Sonra sütunlarda hangi harfler oluştuğu yukarıdan aşağıya ekrana yazdırılır. Hoca bunu, kelimeler yazıldıktan sonra sütunlardan oluşan kelimeler mantıklı mı diye görmek olarak anlattı. Bir karakter matrisi tanımlanır: `char dictionary[50][50]`.

6.2 Satıra Kelime Okumak

Kelime sayısı `%d` ile, kelimeler bir `for` içinde `%s` ile okunur.

Soru: `scanf`'te matrisin neresini yazacağım? **Cevap:** `dictionary[i]`. Bunun açık yazımı `&dictionary[i][0]`'dır. Biri `i`. satırın adresi, diğeri yine `i`. satırın ilk gözünün adresidir. İkisi aynı şeydir.

Soru: Satıra tek seferde kelime alabiliyoruz. Sütuna da alabilir miyiz? **Cevap:** Hayır. Zaten bu program bu yüzden yazılıyor. Sütunlardaki harfler de bir bütün hâlde kelime gibi yazdırılamaz, tek tek yazdırılmaları gerekir.

Soru: Tanımlanmamış `kelimeSayisi` kullandınız. **Cevap:** Hoca kodu yazıp değişken adlarını belirlediğini, sonra yukarı çıkıp hepsini tanımladığını söyledi. Öğrenciler de böyle çalışabilir.

6.3 Sütunları Yazdırma Fikri

Soru: Yukarıda `scanf` ile yazdığımızın aynısı `printf` ile yapılamaz mı? **Cevap:** Hayır, sütunlar satırlar gibi yazdırılamaz. Birinci sütun yukarıdan aşağıya tek tek basılır.

Sınıfla önce `i` satır boyunca 0'dan `kelimeSayisi`'na giden bir döngü ve içinde `%c` ile yazdırma kuruldu. Hoca sütun indisi için bir `j` koydu ve bir dış döngü gerektiğini gösterdi. Sonra soru şu oldu: `j` kaça kadar gidecek?

Hoca, kelimelerin farklı uzunlukta olduğunu göstermek için yorum satırına altı kelimelik bir örnek yazdı. Sütunları tek tek sayarak ilk sütunun A, A, M, B, B, Z, sonrakinin L, E, E, E, E olacağını, kısa kelimelerin bittiği yerde sütunun kısalacağını, `ali`'nin dördüncü gözünde null karakteri bulunduğunu, son sütunlarda yalnız uzun kelimelerin kaldığını anlattı.

Kural yalnızca harflerin olduğu kısımları almaktır. Kaçıncı sütuna kadar gidileceğini belirlemek için en uzun kelimenin uzunluğu (`max`) bulunmalıdır.

6.4 En Uzun Kelimeyi Bulmak

`max` sıfırlanır. `i` ile her kelime gezilir. Her kelime için bir `while`, null karakteri görene kadar `j`'yi artırır. Döngüden çıkınca `j`, `max`'tan büyükse `max`'a atanır.

Soru: Dıştaki `for`'u çıkarıp `while` içinde `i`'yi artırsak olmaz mı? **Cevap:** Olmaz. Bütün kelimeleri gezmek gerekir, bunu yapmanın başka şansı yok.

Soru: Uzunluğa bakarken null karakterini sayıyor muyuz? **Cevap:** Hayır, saymıyoruz.

Soruyu soran öğrenci, atamadan önce `j`'yi bir azaltmayı önerdi. Hoca bunu "doğru olabilir" diyerek ekledi ve hızlı yazdığını, bir yanlış varsa birlikte bulacaklarını söyledi. Bu öneri aşağıda görülecek hatalardan biridir.

6.5 Hata Ayıklama

Hoca kodu bilinçli olarak hızlı yazdı. Sırayla şunlar yaşandı, girdi hep `ali`, `sel`, `ilk` idi.

Birinci çalıştırma: Hiçbir çıktı yok. Önce eksik bir parantez düzeltildi. Sonra programı çalıştırınca hiçbir şey yazmadı. Bir öğrenci, iç `for` döngüsünde `i` ile `j`'nin yerinin yanlış yazıldığını buldu. `i`, 0'dan kelime sayısına gittiğine göre satırı, `j` ise sütunu göstermelidir. Doğrusu `dictionary[i][j]`'dir. Başka bir öğrenci döngü sınırlarını da değiştirmeyi önerdi, ama bu öneri düzeltmeden önceki duruma göreydi. Hoca sorunun ortadan kalktığını söyledi, sınırlar değişmedi.

İkinci çalıştırma: Hâlâ sorun var. Bir öğrenci, `max` hesabındaki ilk `for`'da birden fazla satır olduğu hâlde süslü parantez bulunmadığını söyledi. Parantezsiz `for` yalnız `j = 0` satırını tekrarlıyor, `while` döngüsü bir kez çalışıyordu.

> **Hocanın vurgusu:** Çok önemli bir nokta. Tek bir işlem olsa da süslü parantez kullanalım demiştik. Buna dikkat etmekte fayda var.
>
> **Ders sonrası not:** Bu hatalı sürüm yeniden kurulup denendi. `gcc -Wall` bu `for` için `misleading-indentation` uyarısı verdi, program da derste olduğu gibi hiçbir

sütun yazmadı. Sebep büyük olasılıkla `max`'ın 0'da kalmasıdır: `for` bitince `i`, kelime sayısına eşittir, `while` boş bir satıra bakar. Parantez unutmayı derleyici uyarısıyla da yakalamak mümkündür.

Üçüncü çalıştırma: `max` 2 çıkıyor. Hoca parantezleri ekleyip kontrol amacıyla `max`'ı ekrana yazdırdı. Her sütundan sonra da `\n` koydu. Çıktı `max` için 2 gösterdi. Bir öğrenci, koşul `j < max` olduğundan `j`'yi azaltan satırın çıkarılması gerektiğini söyledi. Başka bir öğrenci, C'de indis 0'dan başladığı için 2'nin doğru olduğunu savundu. Hoca bu düşünceyi şöyle cevapladı: Boyut 3'tür ve `max` 3 hesaplanmalıdır ki döngü 2'ye kadar gitsin.

Üç harfli kelimede `max = 3` iken `j` 0, 1, 2 değerlerini alır. `max = 2` olsaydı son sütun hiç yazılmazdı. Denemede `max = 2` sürümü yalnız `asi` ve `lel` satırlarını verdi.

Dördüncü çalıştırma: Doğru. Satır azaltma kaldırılınca çıktı doğru oldu:

`j`	`ali[j]`	`sel[j]`	`ilk[j]`	Yazılan
0	a	s	i	`asi`
1	l	e	l	`lel`
2	i	l	k	`ilk`

Soru: İç içe döngüde önce satırlar, içeride sütunlar gitmesi gerekmez mi? **Cevap:** Burada önce birinci sütun boyunca satırlar inilir, sonra ikinci sütun boyunca. Sütun yazdırıldığı için olması gereken budur: `j` dışta, `i` içtedir.

6.6 Dersin Son Hâli

Aşağıda hocanın düzeltmelerden sonraki kodu vardır. Hoca karakter karşılaştırmasında burada da `NULL` yazmıştı, önceki uyarıdan kaçınmak için `'\0'` kullanıldı. Bu sürüm `gcc -Wall` ile uyarısız derlendi.

```c
#include <stdio.h>

int main()
{
    char dictionary[50][50];
    int i, j, kelimeSayisi, max;

    printf("Kelime sayisini giriniz\n");
    scanf("%d", &kelimeSayisi);

    printf("Kelimeleri giriniz\n");
    for (i = 0; i < kelimeSayisi; i++) {
        scanf("%s", dictionary[i]);
    }

    max = 0;
    for (i = 0; i < kelimeSayisi; i++) {
        j = 0;
        while (dictionary[i][j] != '\0') {
            j++;
        }
        if (j > max) {
            max = j;
        }
    }
    printf("max: %d\n", max);   /* kontrol amacli */

    for (j = 0; j < max; j++) {
        for (i = 0; i < kelimeSayisi; i++) {
            if (dictionary[i][j] >= 'a' && dictionary[i][j] <= 'z') {
```

```
                printf("%c", dictionary[i][j]);
            }
        }
        printf("\n");
    }
    return 0;
}
```

6.7 Açık Kalan Sorun: Kelimenin Bittiği Yerden Sonrası

Kelimeler farklı uzunlukta olunca program bir sorunla karşılaşır. Hoca `ali`, `mehmet`, `sureyya`, `veli` örneğini verdi: Uzunluklar 3, 6, 7, 4 olduğu için `max` 7'dir. Son sütuna gelindiğinde yalnız `sureyya`'nın `a` harfi yazılmalıdır. Ama döngü diğer kelimelerin aynı gözüne de bakar.

Program yalnız `a` ile `z` arasındaki karakterleri yazdırır. O gözde bu aralıkta bir değer varsa ekrana basılır. Hocaya göre bu yazımın sıkıntısı, olmayan karaktere erişmeye çalışmasıdır. Bir öğrenci de doğru çıktının o an için tesadüf olduğunu söyledi.

Hoca çözümü bir sonraki derse bıraktı ama sınıftan önerileri aldı:

- Matrisin bütün gözlerine başlangıçta bir değer atamak (hoca: güzel fikir, olmayacak bir şey değil).
- Her kelimenin uzunluğunu, satırın ilk gözünde saklamak. Kelimenin uzunluğu sütundan küçükse o gözden değer alınmaz.

  Ders sonrası not: Sorunun canlı kanıtı denemede görüldü. Yukarıdaki kod `ali`, `mehmet`, `sureyya`, `veli` için bir çalıştırmada altıncı sütunda `ty` yerine `rty` yazdı, yani kelimelerde o konumda olmayan bir harf bastı. Hocanın "olmayan elemana erişme" dediği durum büyük olasılıkla dizinin dışına çıkmak değil, matrisin içinde olup hiç değer atanmamış bir gözü okumaktır. Sonuç çalıştırmadan çalıştırmaya değişti: Tekrarlarda aynı sütun `tyk`, `ty` ve `vty` olarak da çıktı.

Birinci öneri `char dictionary[50][50] = {{0}};` ile uygulanıp yazdırma koşulu `dictionary[i][j] != '\0'` yapıldığında kod uyarısız derlendi ve doğru sonucu verdi:

Sütun (j)	Yazılan
0	`amsv`
1	`leue`
2	`ihrl`
3	`mei`
4	`ey`
5	`ty`
6	`a`

Dersin Toparlaması

Dersin ilk yarısında dönem projesi olan Kızma Birader oyununun kuralları, tahtası ve kullanıcıyla etkileşimi konuşuldu. Ana mesaj tasarımı koddan önce yapmaktır: Rota çok sayıda `if` yerine eşleme tablosuyla izlenebilir, piyon durumları tutulmalı, problem modüllere bölünmeli, her adımda tahta gösterilmeli ve önceki pozisyon saklanmalıdır.

İkinci yarıda iki küçük C programı birlikte yazıldı. Kelime karşılaştırmada null karakterinin her harften küçük olması, sütun yazdırmada satır ve sütun indislerinin ayrımı, süslü parantezlerin önemi ve kontrol amaçlı `printf` ile hata ayıklama görüldü. Kısa kelimelerin bittiği yerden sonrasını okuma sorunu bir sonraki derse bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.